

TALLER AVANZADO: SOLUCIÓN DEL CAMPO ELECTROESTÁTICO 2D POR DIFERENCIAS FINITAS

Este taller abordará la resolución de la Ecuación de Laplace o la Ecuación de Poisson en dos dimensiones, lo cual es fundamental en el diseño de componentes electrónicos y dispositivos de alto voltaje.

1. Tema del Proyecto: Potencial Electroestático en una Región Cuadrada

El software calculará la distribución del Potencial Eléctrico $V(x, y)$ en una región 2D delimitada por condiciones de contorno fijas (voltajes conocidos en las fronteras).

La Ecuación a Resolver (Ecuación de Laplace en 2D):

Implementación Numérica: El Método de Diferencias Finitas (MDF)

Al aplicar el MDF a una cuadrícula con paso h , la Ecuación de Laplace en un punto interior (i, j) se aproxima por:

$$V_{i+1,j} + V_{i-1,j} + V_{i,j+1} + V_{i,j-1} - 4 V_{i,j} = 0$$

Funcionalidades Clave:

1. Discretización: Crear una matriz de $N \times N$ puntos para representar la región 2D.
2. Resolución (Método Iterativo): Implementar un método iterativo, como el Método de Jacobi o el Método de Gauss-Seidel, para resolver el sistema de ecuaciones, convergiendo la matriz V a la solución.
3. Cálculo del Campo Eléctrico (E): Una vez que $V(x, y)$ converge, calcular el campo eléctrico usando el gradiente: $E = -\text{grad } V$. Esto requiere la diferenciación numérica de la matriz V .
4. Generación de Gráficos: Visualizar el potencial (V) como un mapa de colores (heatmap) y el campo eléctrico (E) como un gráfico de vectores (quiver plot).

2. Estructura y Tecnología Refinada

2.1. Backend (El Módulo Científico - Uso de numpy y scipy)

El *backend* debe manejar las operaciones matriciales y la lógica iterativa de la simulación.

- Nombre del Módulo: `campo_estatico_mdf`.
- Requisito clave: La clase principal, por ejemplo `LaplaceSolver2D`, debe utilizar `numpy` intensivamente para la manipulación de matrices y, opcionalmente, `scipy.sparse` si se usa una solución directa de matrices dispersas (más avanzado).
- Contenido:
 - Clase `LaplaceSolver2D`:

- Método para inicializar la malla y las condiciones de contorno (voltajes fijos en los bordes).
- Método `resolver_jacobi` (o Gauss-Seidel) que implementa el algoritmo iterativo y detiene la iteración cuando la diferencia máxima entre pasos es menor a una tolerancia (ϵ).
- Método `calcular_campo_e` que usa `numpy.gradient` o diferenciación centrada para obtener E_x y E_y .
- Distribución: Publicación obligatoria en PyPI.

2.2. Frontend (Interfaz Streamlit)

- Tecnología: Streamlit.
- Contenido:
 - Inputs: Recibir el tamaño de la malla $N \times N$, los voltajes de contorno (ej. Izquierda=0V, Derecha=10V), y la tolerancia (ϵ).
 - Procesamiento: Importar `campo_estatico_mdf` y ejecutar la simulación.
 - Outputs:
 - Mapa de Colores (Heatmap): Distribución de potencial $V(x, y)$ (usando `matplotlib.pyplot` o `Plotly`).
 - Gráfico de Vectores: Campo Eléctrico $E(x, y)$ (usando `matplotlib.pyplot.quiver`).
 - Métrica de Convergencia: Mostrar el número de iteraciones requeridas para alcanzar la tolerancia.

3. Pruebas Unitarias y Documentación

3.1. Pruebas Unitarias

Las pruebas (unittest o pytest) deben validar:

- **Caso Trivial:** Probar un caso donde la solución es obvia (ej. todas las fronteras a 0V, el resultado debe ser $V=0$ en toda la región interior).
- **Convergencia:** Probar que el método `resolver_jacobi` converge para un conjunto simple de condiciones de contorno (ej. dos lados a 0V y dos lados a 10V) dentro de un número razonable de iteraciones.
- **Cálculo del Campo:** Verificar que el cálculo del campo eléctrico es correcto para un potencial conocido (ej. si V es lineal, E debe ser constante).

3.2. Documentación con Sphinx + GitHub Pages

- La documentación debe explicar la fórmula de diferencias finitas (la aproximación de la Ecuación de Laplace) y el algoritmo iterativo (Jacobi/Gauss-Seidel).
- Referencia de la API del paquete `campo_estatico_mdf` generada automáticamente.
- Publicación en GitHub Pages.

REQUERIMIENTOS FUNCIONALES

RF1: Módulo de Cálculo (Backend)

ID	Requisito Funcional	Descripción
RF1.1	Discretización de Malla	El sistema debe ser capaz de inicializar una matriz de $N \times N$ puntos que represente la región 2D, aplicando las condiciones de contorno de voltaje fijo en los bordes.
RF1.2	Resolución Numérica	El sistema debe implementar un método iterativo (Jacobi o Gauss-Seidel) para resolver numéricamente la Ecuación de Laplace usando la aproximación del Método de Diferencias Finitas (MDF).
RF1.3	Criterio de Convergencia	La iteración de la solución debe detenerse cuando la diferencia máxima entre los valores de potencial V de dos pasos sucesivos sea menor a una tolerancia (ϵ) definida.
RF1.4	Cálculo del Campo E	El sistema debe calcular las componentes del Campo Eléctrico $E = -\text{grad } V$ a partir de la distribución de potencial V convergida, utilizando diferenciación numérica (ej. <code>numpy.gradient</code>).
RF1.5	Interfaz de Módulo	El Backend debe exponer una Clase <code>LaplaceSolver2D</code> con métodos para inicializar la malla, aplicar condiciones de contorno, y ejecutar la simulación iterativa (<code>resolver_jacobi</code>).

RF2: Interfaz de Usuario (Frontend Streamlit)

ID	Requisito Funcional	Descripción
RF2.1	Captura de Inputs	La interfaz debe permitir al usuario ingresar los parámetros de simulación: el tamaño de la malla ($N \times N$), los voltajes de contorno (ej. $V_{\text{izquierda}}$, V_{derecha}), y la tolerancia de convergencia (ϵ).
RF2.2	Visualización de Potencial	El sistema debe generar y mostrar un Mapa de Colores (Heatmap) de la distribución del potencial eléctrico $V(x, y)$ resuelto.
RF2.3	Visualización del Campo E	El sistema debe generar y mostrar un Gráfico de Vectores (Quiver Plot) para visualizar la dirección y magnitud del campo eléctrico $E(x, y)$.
RF2.4	Reporte de Métrica	La interfaz debe mostrar el número de iteraciones que el algoritmo necesitó para alcanzar la tolerancia de convergencia (ϵ).

REQUERIMIENTOS NO FUNCIONALES

RNF1: Arquitectura y Tecnologías

ID	Requisito No Funcional	Descripción
RNF1.1	Modularidad	El proyecto debe mantener una clara separación entre el Backend (lógica científica) y el Frontend (interfaz de usuario).
RNF1.2	Tecnología Backend	El Backend debe implementarse en Python y utilizar obligatoriamente las bibliotecas <code>numpy</code> y, preferiblemente, <code>scipy.sparse</code> para la eficiencia en operaciones matriciales.
RNF1.3	Tecnología Frontend	El Frontend debe construirse utilizando el <i>framework</i> <code>Streamlit</code> para la interfaz web.
RNF1.4	Bibliotecas de Visualización	La generación de gráficos debe usar bibliotecas estándar de Python como <code>matplotlib</code> , <code>pyplot</code> o <code>Plotly</code> .

RNF2: Pruebas y Calidad

ID	Requisito No Funcional	Descripción
RNF2.1	Framework de Pruebas	Las pruebas unitarias deben implementarse utilizando <code>unittest</code> o <code>pytest</code> .
RNF2.2	Cobertura de Pruebas	Se deben incluir pruebas para validar la correcta inicialización de las condiciones de contorno y, específicamente, la prueba del Caso Trivial ($V=0$) y la prueba de Convergencia para casos simples conocidos.
RNF2.3	Precisión de Campo E	Las pruebas deben verificar la correcta implementación del cálculo del campo eléctrico E (ej. verificando la solución lineal en casos donde V es lineal).

RNF3: Despliegue, Documentación y Colaboración

ID	Requisito No Funcional	Descripción
RNF3.1	Distribución del Backend	El módulo del Backend (<code>campo_estatico_mdf</code>) debe ser empaquetado y publicado en PyPI (o Test PyPI).
RNF3.2	Control de Versiones	El proyecto debe alojarse en un repositorio de GitHub con evidencia de trabajo colaborativo (múltiples <i>commits</i> y <i>Pull Requests</i> de diferentes cuentas).
RNF3.3	Documentación Técnica	Se debe generar documentación técnica usando Sphinx que explique el MDF, el algoritmo iterativo, y contenga una referencia de la API generada automáticamente.
RNF3.4	Despliegue de Documentación	La documentación generada por Sphinx debe estar accesible a través de GitHub Pages.

Categoría de Evaluación	Criterios Clave a Evaluar	Ponderación	Nivel 4 (Excelente: 86-100%)	Nivel 2 (Aceptable: 50-85%)	Nivel 0 (Insuficiente: <50%)	VALORACIÓN
I. Solución Científica y Precisión	(40 Pts)	Lógica Numérica: Correcta implementación del Método de Diferencias Finitas (MDF) y el algoritmo iterativo (Jacobi/Gauss-Seidel). Precisión: La solución converge a la tolerancia y el cálculo del Campo E es correcto y validado.	El código resuelve la Ecuación de Laplace con alta precisión, el algoritmo converge eficientemente y el cálculo de E es científicamente riguroso.	La lógica MDF está presente, pero presenta errores menores en la convergencia, la implementación del algoritmo es ineficiente o el cálculo de E es impreciso.	Falla en implementar o resolver el MDF. La solución es incorrecta o el programa no converge.	
II. Arquitectura y Modularidad	(25 Pts)	Separación RNF1.1: Clara división entre Backend (<code>campo_estatico_mdf</code>) y Frontend (Streamlit). Distribución RNF3.1: Backend publicado correctamente en PyPI. Tecnología RNF1.2: Uso efectivo de <code>numpy</code> y <code>scipy</code> .	Perfecta separación modular. La lógica científica está encapsulada en la clase <code>LaplaceSolver2D</code> del Backend, que está publicado en PyPI y utiliza las librerías científicas requeridas de forma óptima.	Existe separación, pero el Backend no está bien encapsulado (ej. no usa clases) o la publicación en PyPI tiene errores.	No hay separación modular clara, o el Backend no fue publicado en PyPI.	
III. Interfaz de Usuario (Streamlit)	(15 Pts)	RF2: La interfaz Streamlit es completa, captura todos los inputs requeridos (N , V_{contorno} , ϵ) y genera los dos tipos de gráficos	La interfaz Streamlit es funcional, pero le faltan algunos <i>inputs</i> o uno de los dos tipos de gráficos requeridos.	La interfaz es inusable, no utiliza Streamlit, o no genera ningún gráfico.		

		necesarios: Heatmap (\$V\$) y Quiver Plot (\$\vec{E}\$).				
IV. Calidad del Código y Pruebas Unitarias	(10 Pts)	RNF2.2: Cobertura de pruebas unitarias (<code>unittest/pytest</code>) que validan la inicialización, la convergencia y la precisión del Campo E (incluyendo casos triviales).	Se incluyen pruebas unitarias, pero la cobertura es limitada o fallan en validar la precisión científica de los casos límite.	El código carece de pruebas unitarias o las existentes están rotas/son triviales.		
V. Documentación y Colaboración	(10 Pts)	RNF3.2-3.4: Evidencia de colaboración (múltiples <i>commits</i> y <i>Pull Requests</i>). Documentación con Sphinx que incluye la Referencia de la API y está accesible vía GitHub Pages.	El historial de Git muestra colaboración. La documentación está presente con Sphinx, pero le falta la Referencia de la API o no está en GitHub Pages.	No hay evidencia de colaboración. La documentación está ausente o no se usó Sphinx.		